

GaussianFormer: per-object adapters and the rasterized-canvas architecture

Week of 8-16 September 2026, with renders

Shahaf Valencia Levy

17 September 2026

1. Storage: what an adapter costs in Gaussians

A Gaussian is $14 \text{ float32} = \mathbf{56 \text{ bytes}}$. A LoRA adapter of rank r over a set of linear maps costs $4 * r * \text{sum}(\text{in} + \text{out})$ bytes, so every adapter has a price in Gaussians:

per-object payload	params	size	= Gaussians
r=1, attention only	119,808	0.48 MB	8,558
r=4, attention only	479,232	1.92 MB	34,231
r=4, attention + FFN (used below)	1,308,672	5.23 MB	93,477
r=16, attention only	1,916,928	7.67 MB	136,923
full fine-tune	196,177,815	784.7 MB	14,012,701
the 20k splat itself		1.12 MB	20,000

Rank is the cheap axis ($r=1$ to $r=16$ is $16\times$); adding FFN targets costs $2.7\times$ at equal rank. The winning adapter is $4.7\times$ the size of the splat it renders, and at equal bytes the rasterizer still wins by about 2.1 dB. So what follows is an **enhancement** result (given a splat, render it better than rasterizing it), not yet a compression result. The $10\times$ smaller rank-1 attention-only adapter has never been tried on the current base.

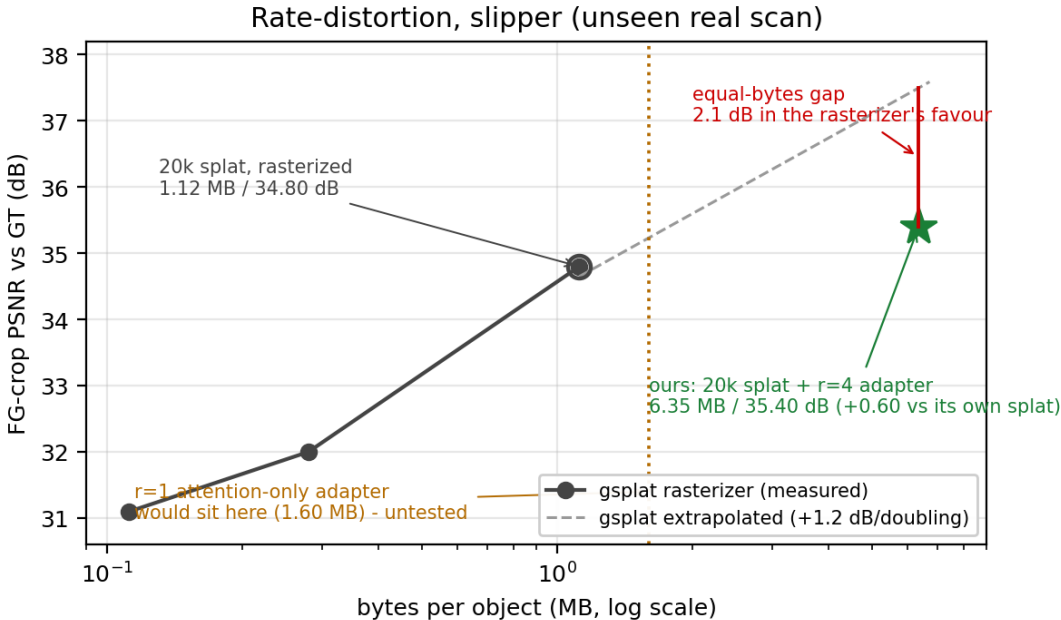


Figure 1: Rate-distortion on the slippier. Green star: our best result. Red bar: what the same bytes buy as Gaussians.

2. Architecture: what we changed and what it bought

Fixed harness: 10 training objects, 30k steps, warm start from V18. Readout = FG-crop PSNR below the rasterizer on the 10 training objects (*fit*) and on 300 unseen objects (*held-out*). Lower is better; negative beats the rasterizer.

What each change is. In the baseline, position enters the network only through a 3-D rotary embedding (RoPE) on Gaussian centres, 12 frequencies spanning 1 to 5 radians per world unit, in both the scene stage and the cross-attention of the view stage; ray tokens carry the camera origin as their position, so the view stage has no notion of where a patch sits in the image and has to infer every projection from 3-D position plus camera.

- **P3, RoPE bandwidth.** Hypothesis: fine detail is lost because attention cannot resolve positions finely enough. Three variants: (a) rotary dim 12 to 32 with positions scaled 4x (band 4 to 20 rad/unit, rotates 48 of 64 channel pairs); (b) a second 3-D band at 8x scale (8 to 40 rad/unit) added in previously unrotated pairs, pretrained pairs untouched; (c) a 2-D RoPE on the patch-grid position for ray-token self-attention. All three are within 0.1 dB of the baseline: positional resolution of attention is not the bottleneck.
- **P2, projected 2-D RoPE in cross-attention.** Every Gaussian is explicitly perspective-projected into the current view. Its projected patch coordinate (u, v) gets a 2-D RoPE on the cross-attention *keys*; each ray token gets its own patch centre on the *queries*. The attention logit between a patch and a Gaussian then carries a term that depends only on their 2-D offset in the image, so attention can prefer Gaussians that land near the patch without learning projection implicitly. The only geometry change that moved both axes.
- **P2 variants.** A 4x sharper 2-D band, 32 rotary dims instead of 16, the same 2-D RoPE also in ray self-attention, and a zero-initialised proximity bias plus depth and footprint features. All flat: the projected coordinate itself is the effect, its resolution is not.
- **P1, rasterized canvas.** The input splat is rasterized with gsplat from the same camera (in $\log_{10}(x+1)$ space) and added to the ray tokens through a zero-initialised linear layer. At load the network is exactly V18; it learns to read the canvas, and the decoder becomes a refiner of it. The largest held-out mover by far, the weakest on fit.
- **Foreground-weighted loss.** Pixels whose target luminance is at most 0.02 (the black background) get weight 0.05 in the loss. Objects cover a minority of each frame, so the loss had been dominated by background that every model already gets right. It moves fit, not generalisation.
- **Cycles.** Warm restarts of the same run with a fresh learning-rate schedule.

The recurring rule: only changes that are *additive and zero-initialised* survive the warm start; anything that alters pretrained function (P3a, the P2 RoPE itself) needs a low-resolution recovery stage first.

change	fit	held-out	verdict
V18 baseline	7.62	20.44	reference
P3: RoPE bandwidth (3 variants)	7.53	20.69	null
P2: projected 2-D RoPE in cross-attention	6.47	19.07	keep
P1: rasterized canvas as decoder input	6.78	17.47	keep
P1 + P2	6.07	16.95	additive
P1 + P2 + foreground-weighted loss (c1)	2.91	17.27	the recipe
same, warm-restarted 4 cycles (c4)	-2.15	17.06	best base
P2 + fg, 3 cycles (best without canvas)	-0.35	20.34	comparison

All steps significant under a cluster bootstrap over scenes (P2 vs baseline -1.38 [-1.47, -1.28]; P1 vs P2 -1.60 [-1.69, -1.51]; c4 fit -2.15 [-3.25, -1.08]). Cycles only stop hurting generalisation with the canvas: P2+fg drifts +0.95 dB on held-out over three cycles, the canvas arm improves on both axes every cycle.

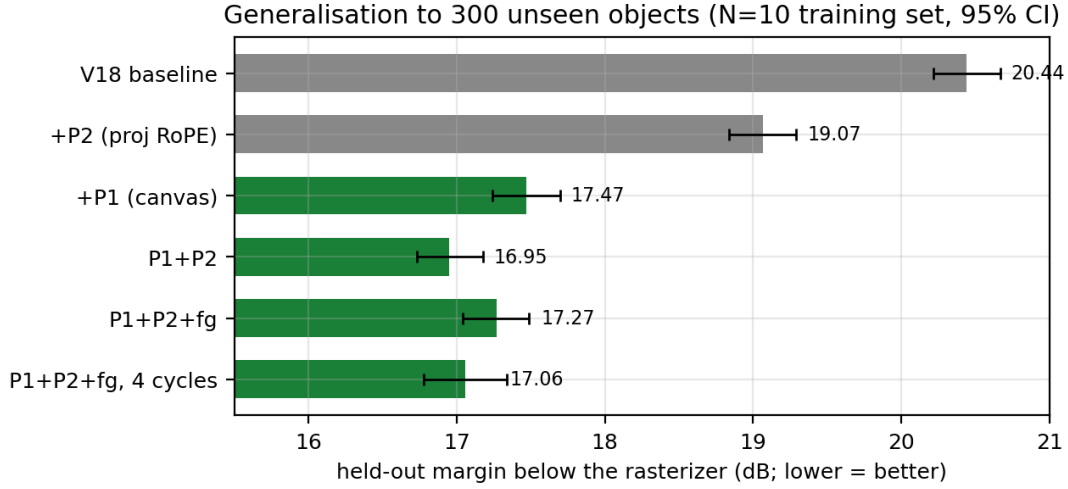


Figure 2: Held-out generalisation by arm, 95% CI.

2.1 What the ladder looks like

Every figure below: **GT** | **rasterizer** | **V18 seed** | **P2+fg c3** | **canvas c4**, on objects the model has never seen, foreground-cropped exactly like the metric. Numbers are PSNR and the gap to the rasterizer; green means the model beats it.

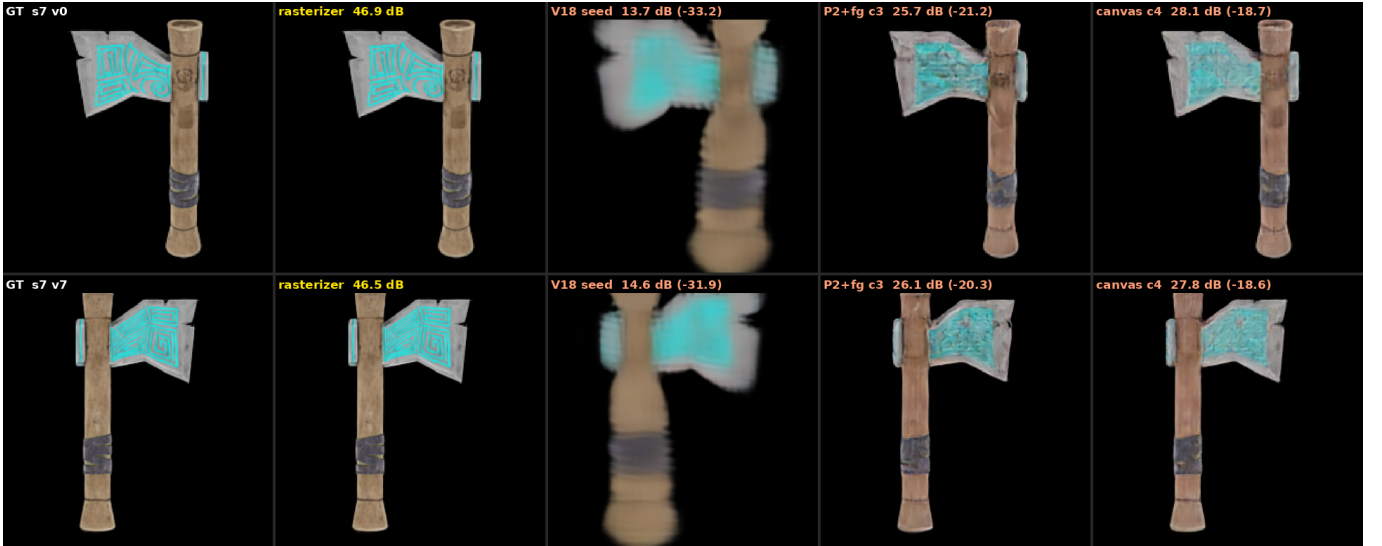


Figure 3: The axe (held-out scene 7). The seed cannot place the blade; P2 recovers shape but invents the etchings; the canvas keeps them legible but still blurs the fine strokes. -18.7 dB below the rasterizer.

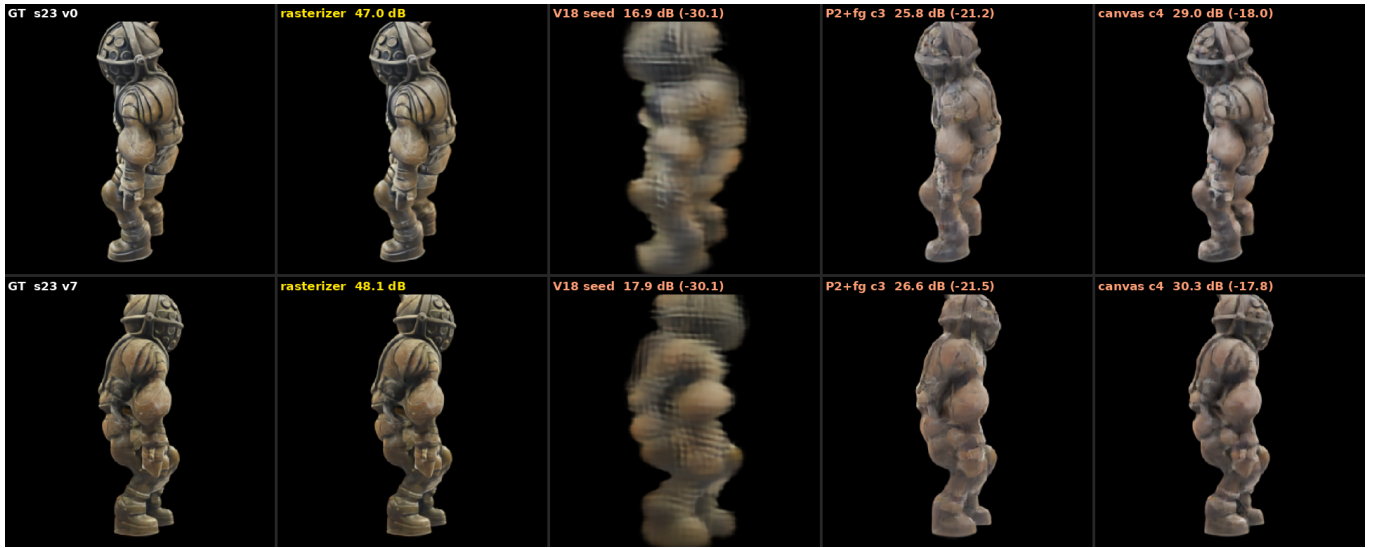


Figure 4: The diver figure (scene 23). Rivets and straps survive only with the canvas.

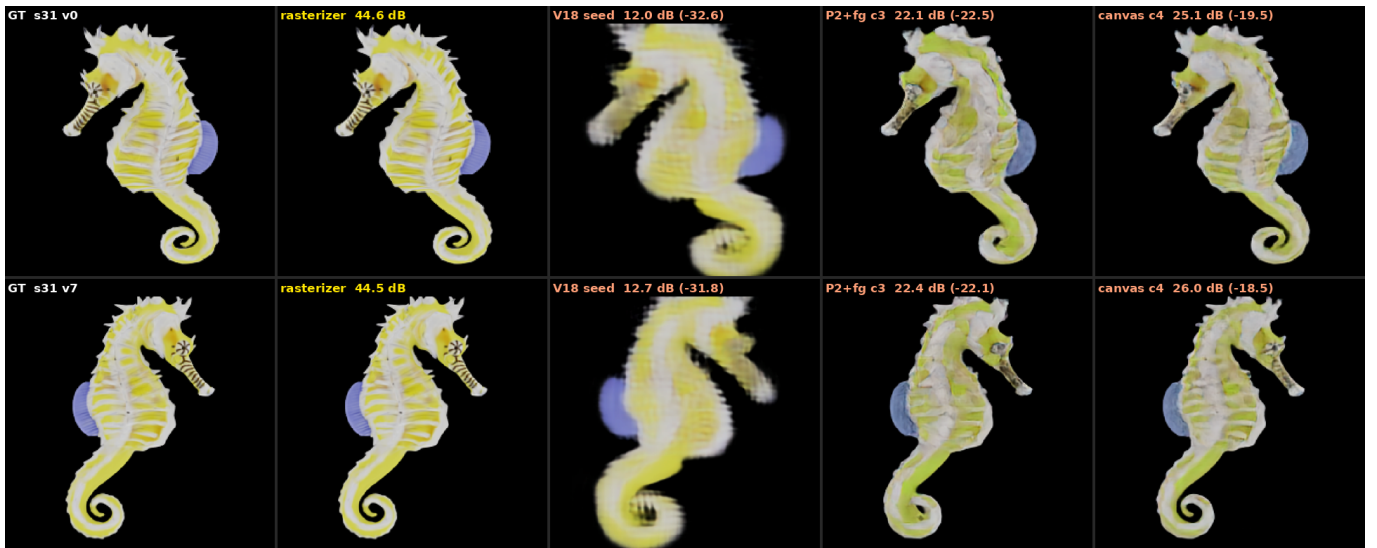


Figure 5: The seahorse (scene 31), the hardest of the four: thin spines and a painted eye.

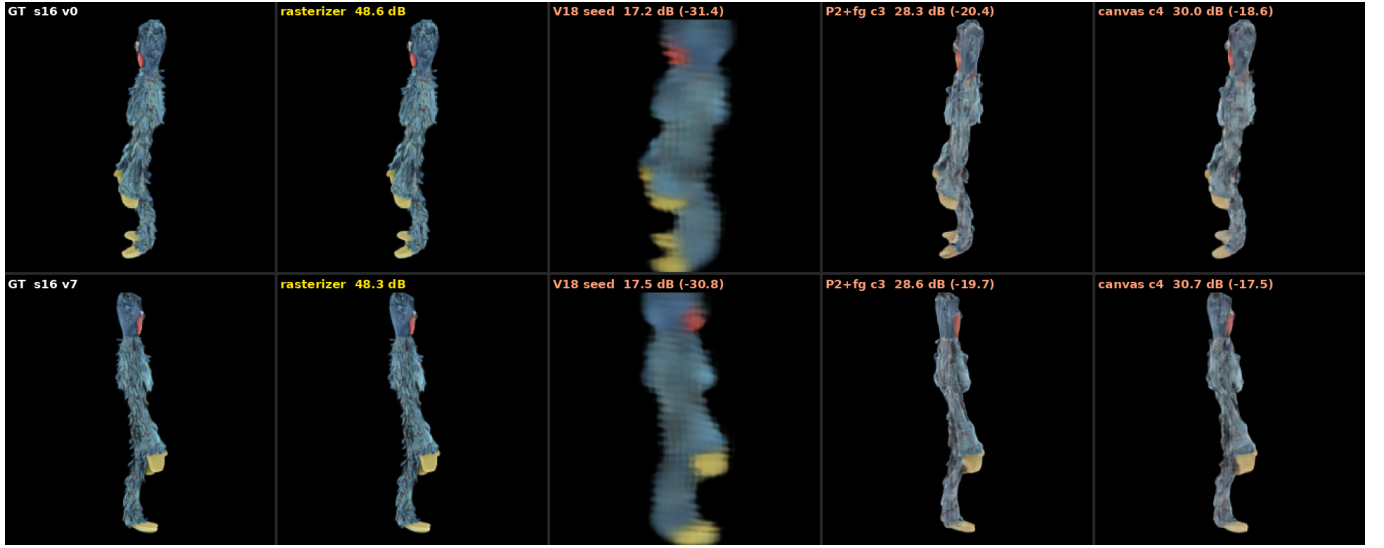


Figure 6: The furry creature (scene 16). Fur is where all three models are furthest from the rasterizer.

Native-resolution 96-px windows (nearest-neighbour upscale, no smoothing) on the same views, with error maps on a shared scale per row:

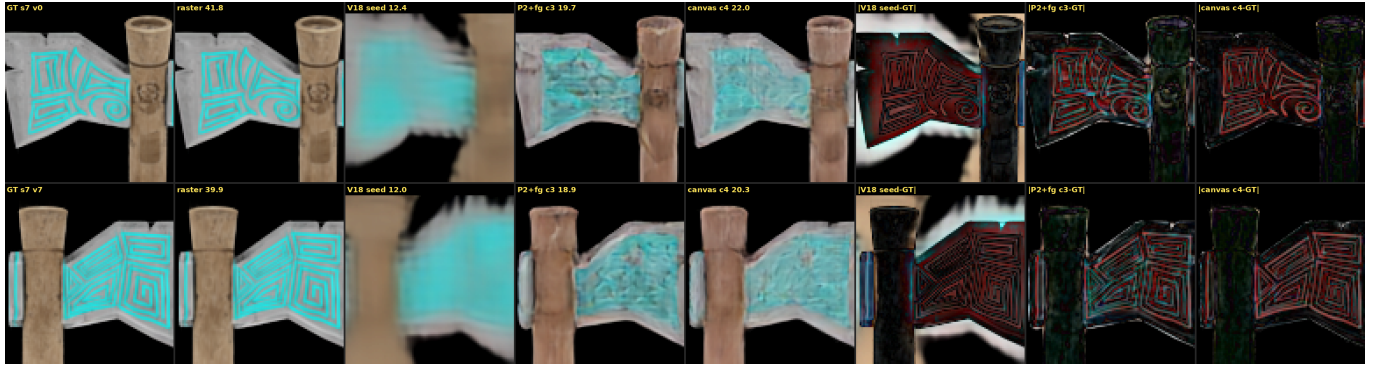


Figure 7: Axe etchings at native resolution. P2 hallucinates strokes; the canvas keeps the true ones but smooths them.



Figure 8: Diver helmet: the canvas output is sharper and its error map is dominated by edges, not by regions.

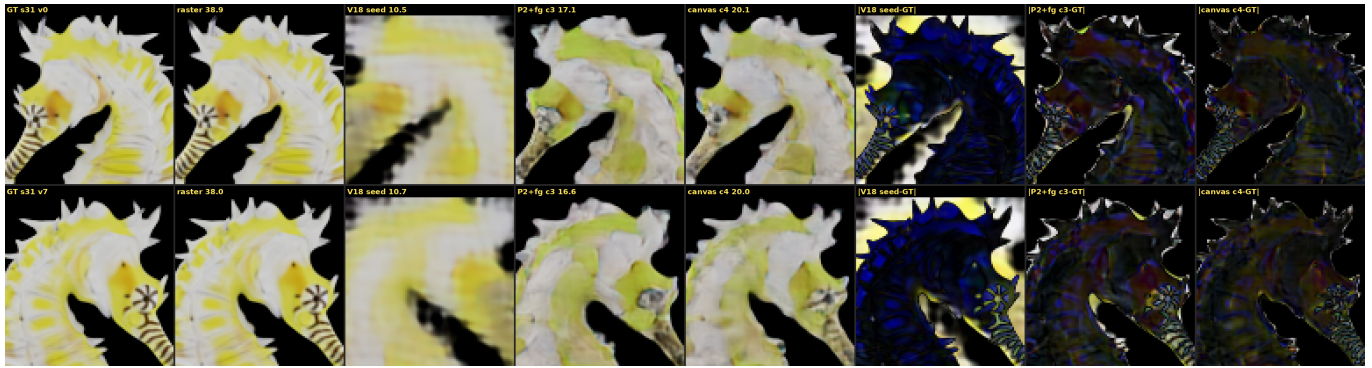


Figure 9: Seahorse spines.

Where the canvas gains most. The four unseen objects with the largest improvement over the same recipe without a canvas (+6 to +9 dB):

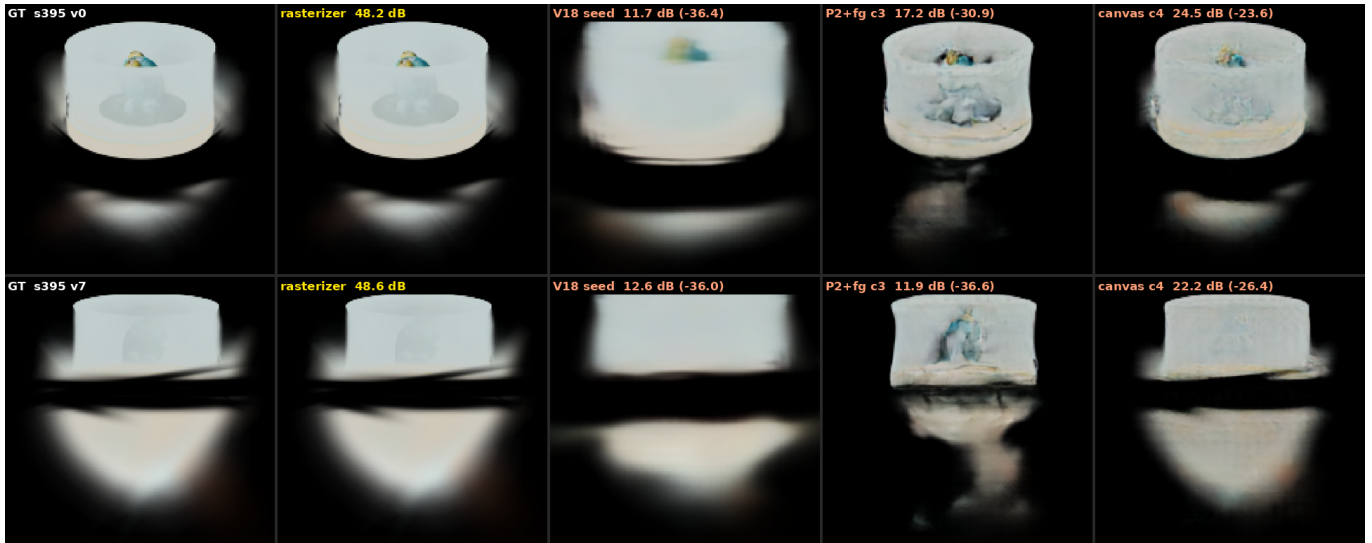


Figure 10: Scene 395: +8.9 dB from the canvas. Without it the model renders a different object.

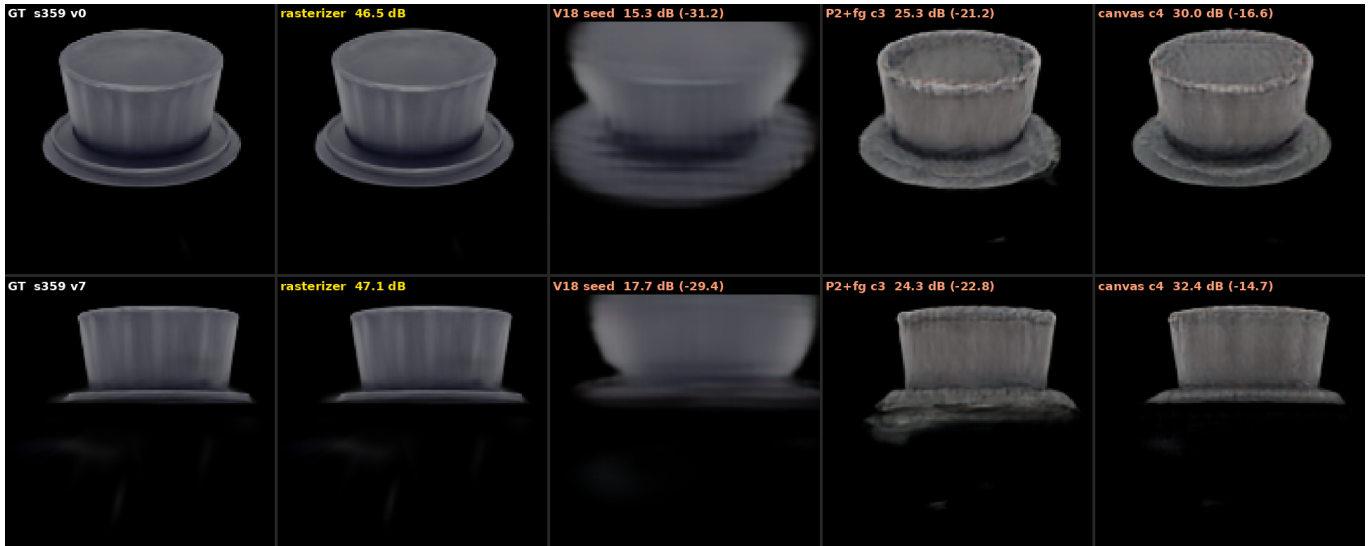


Figure 11: Scene 359: +6.7 dB.

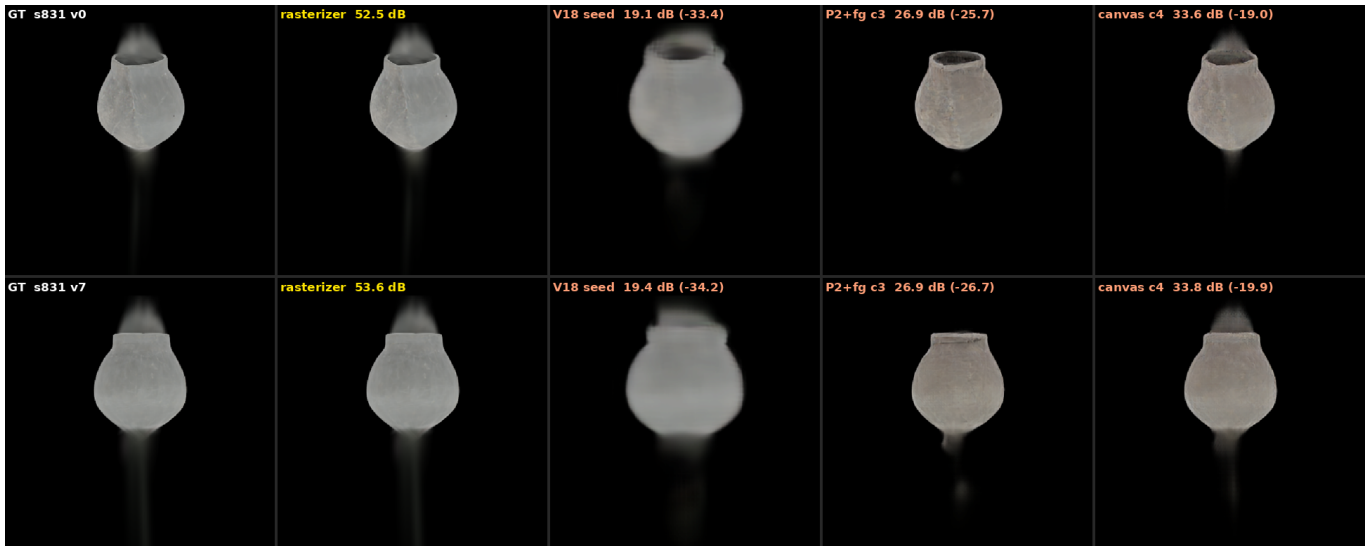


Figure 12: Scene 831: +6.2 dB; the canvas arm reaches 33.6 dB, our best held-out number.

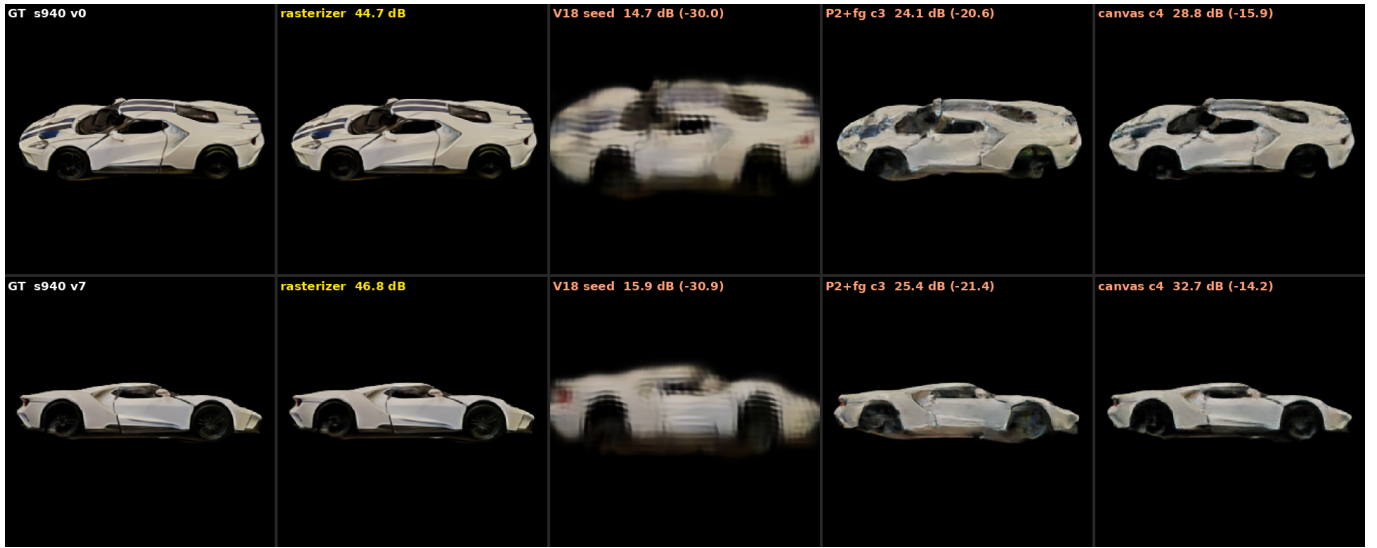


Figure 13: Scene 940: +5.9 dB.

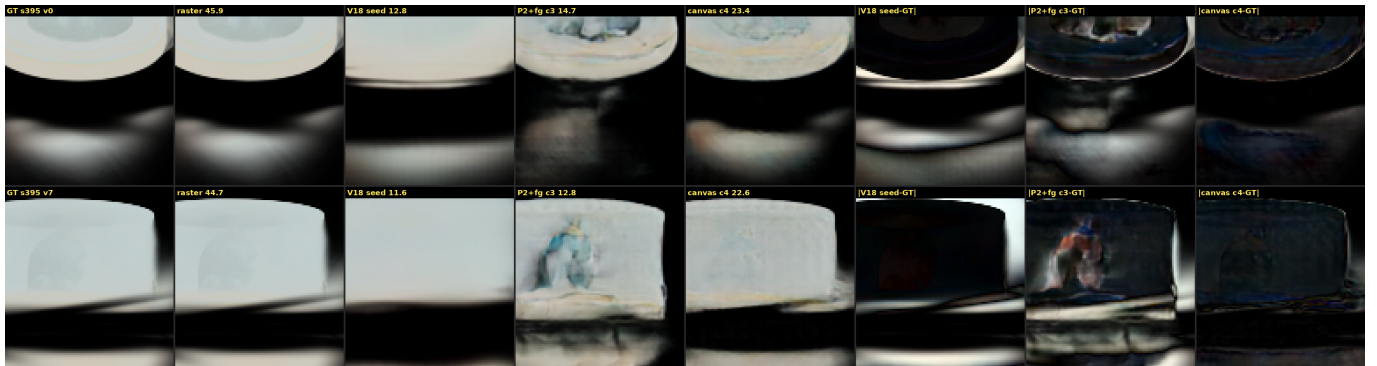


Figure 14: Scene 395 at native resolution.

Where it does not help. The two objects where the canvas arm is no better than the non-canvas one, and one where every model fails the same way:

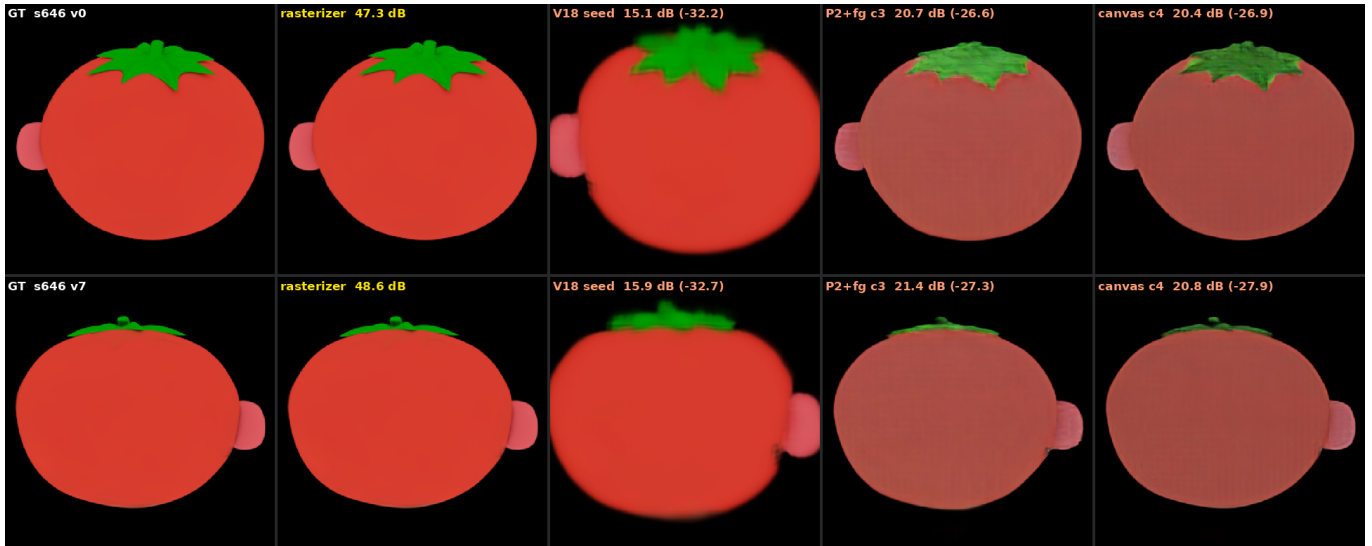


Figure 15: Scene 646: a near-flat object the rasterizer renders at 47 dB; both models lose 27 dB to it, the canvas slightly more.

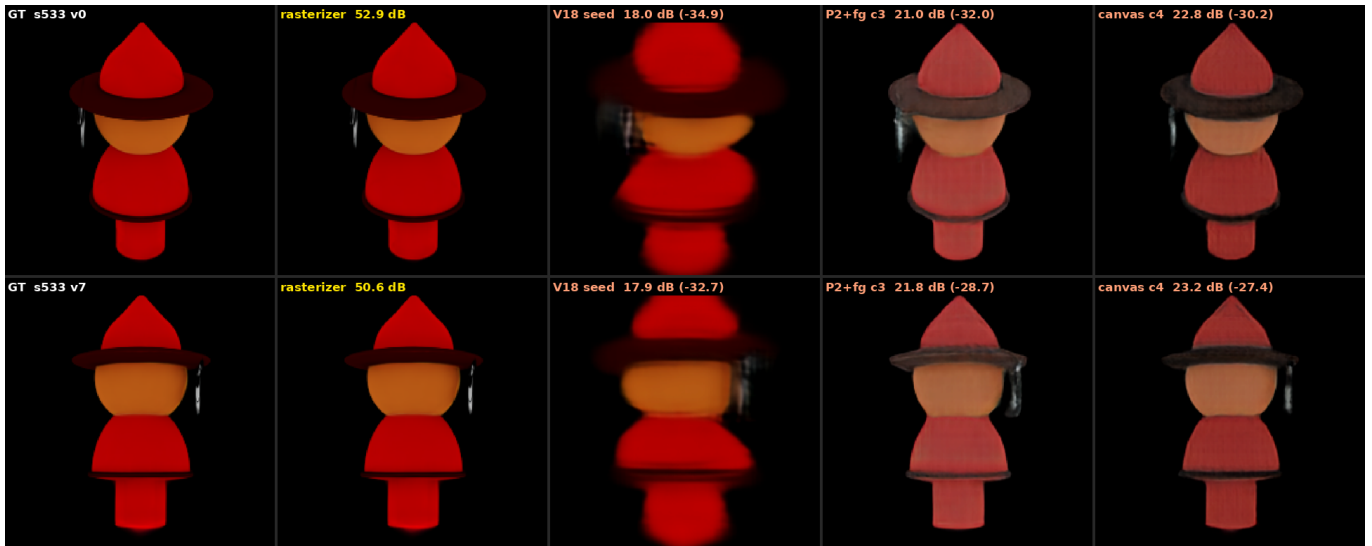


Figure 16: Scene 533: the rasterizer's best object in the set (53 dB); nothing we have comes within 30 dB.

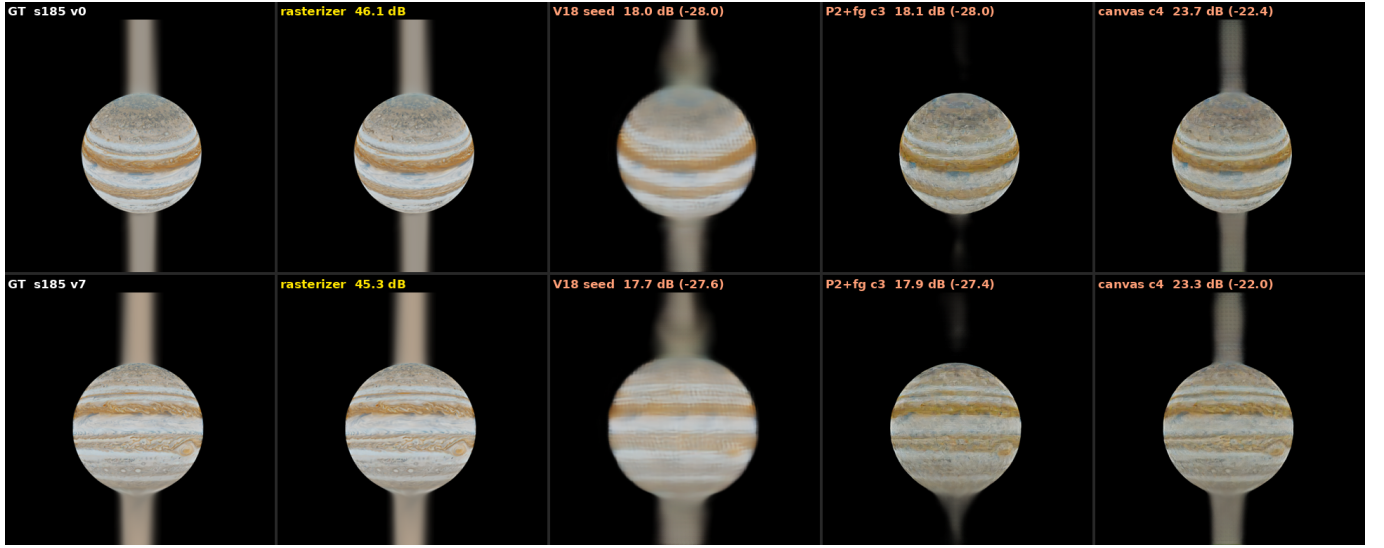


Figure 17: Scene 185: a banded sphere; P2 and the seed tie, the canvas gains 5.5 dB, and all three lose the band boundaries.

2.2 It scales

Same recipe with 100 training objects at equal compute:

arm	fit	held-out
baseline	13.40	17.93
P2 + fg	10.51	16.38
P1 + P2 + fg	7.77	11.25 [11.01, 11.49]

The canvas advantage over the identical recipe without it grows from 2.12 dB at N=10 to 5.13 dB at N=100. The canvas at 100 objects beats the 1000-object baseline (16.70) by 5.5 dB.

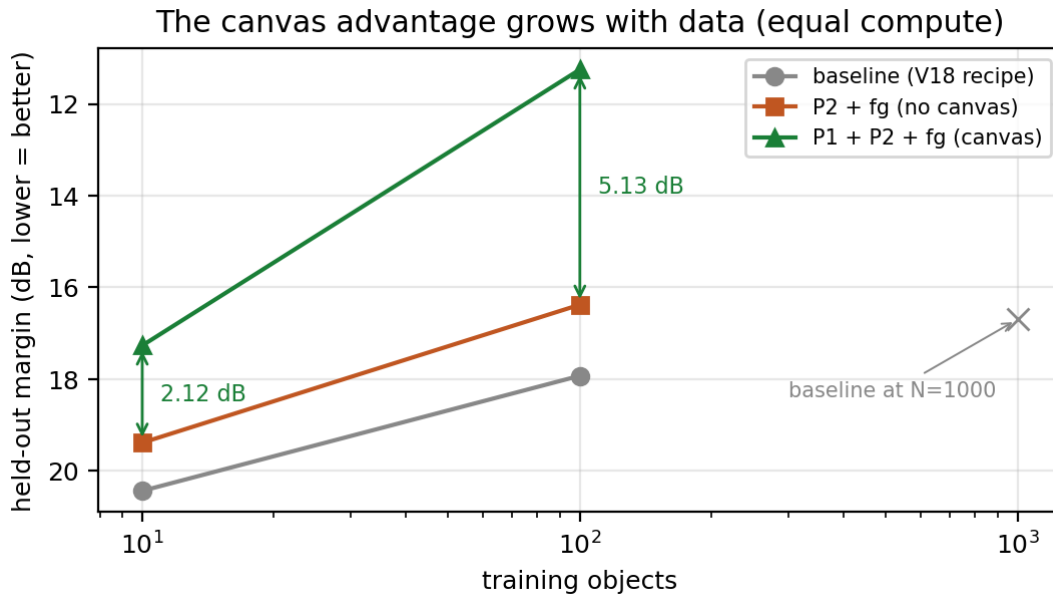


Figure 18: The gap widens with data.

3. The result: a 5.23 MB adapter beats the rasterizer on an unseen scan

Shared base (canvas recipe, 3 cycles, 10 objects) plus a rank-4 adapter trained on one unseen real scan (the slipper), evaluated on views neither saw:

range	ours	rasterizer	delta
novel random	35.95	35.79	+ 0.16 (16/24)
novel close	30.20	28.20	+ 2.00 (8/8)
novel far	38.92	38.39	+ 0.53 (6/8)
all 40 views			+0.60 , 30/40 won

Columns below: **GT** | **rasterizer** | **canvas base with no adapter** | **base + adapter**. The base alone is far behind on a scan it never saw; the adapter is what closes the gap.



Figure 19: Slipper, close range, views 0-3. The base alone loses 3-4 dB to the rasterizer on this unseen scan; the adapter beats the rasterizer on all eight close views by 1.7-2.2 dB.

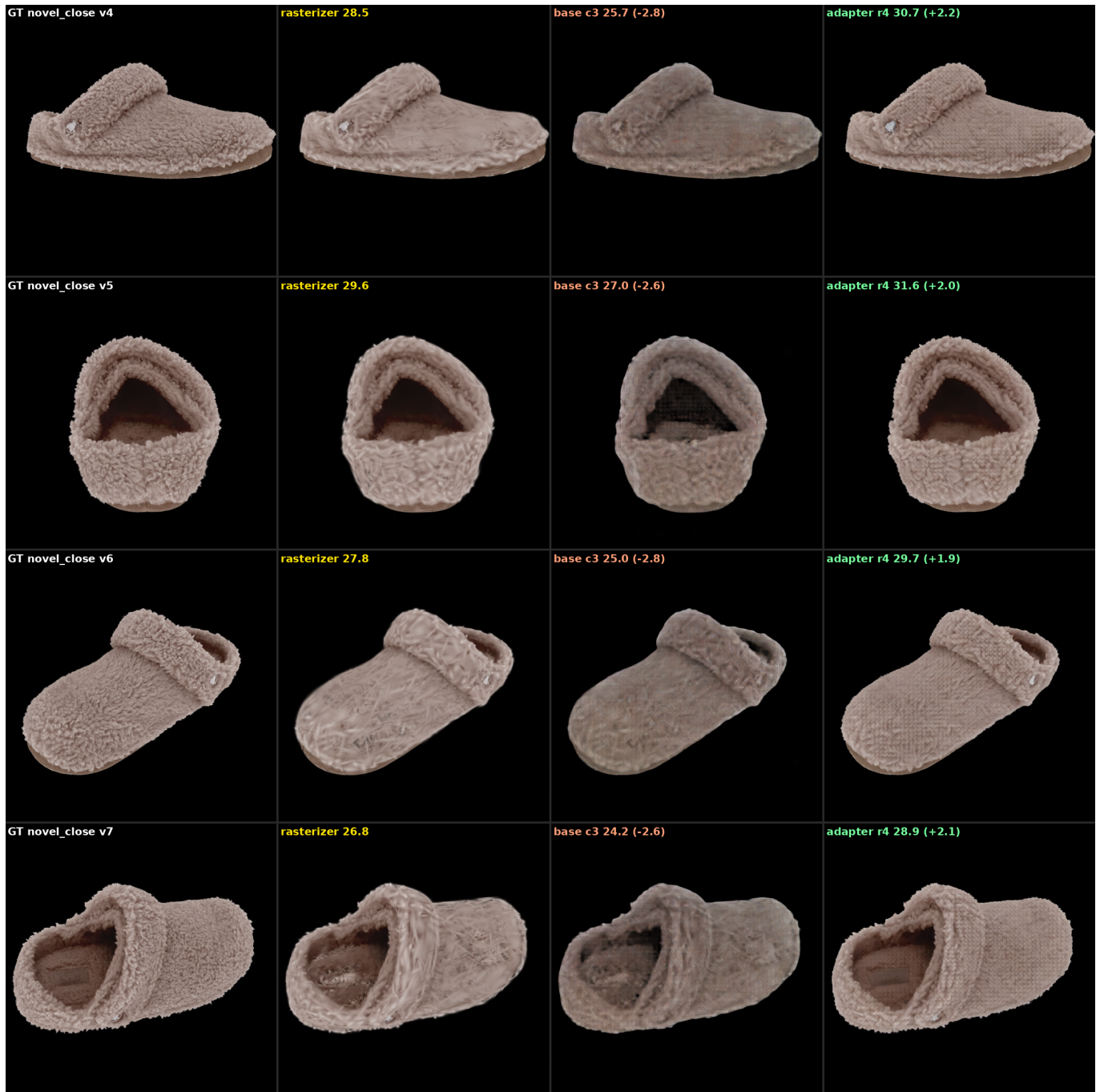


Figure 20: Slipper, close range, views 4-7.

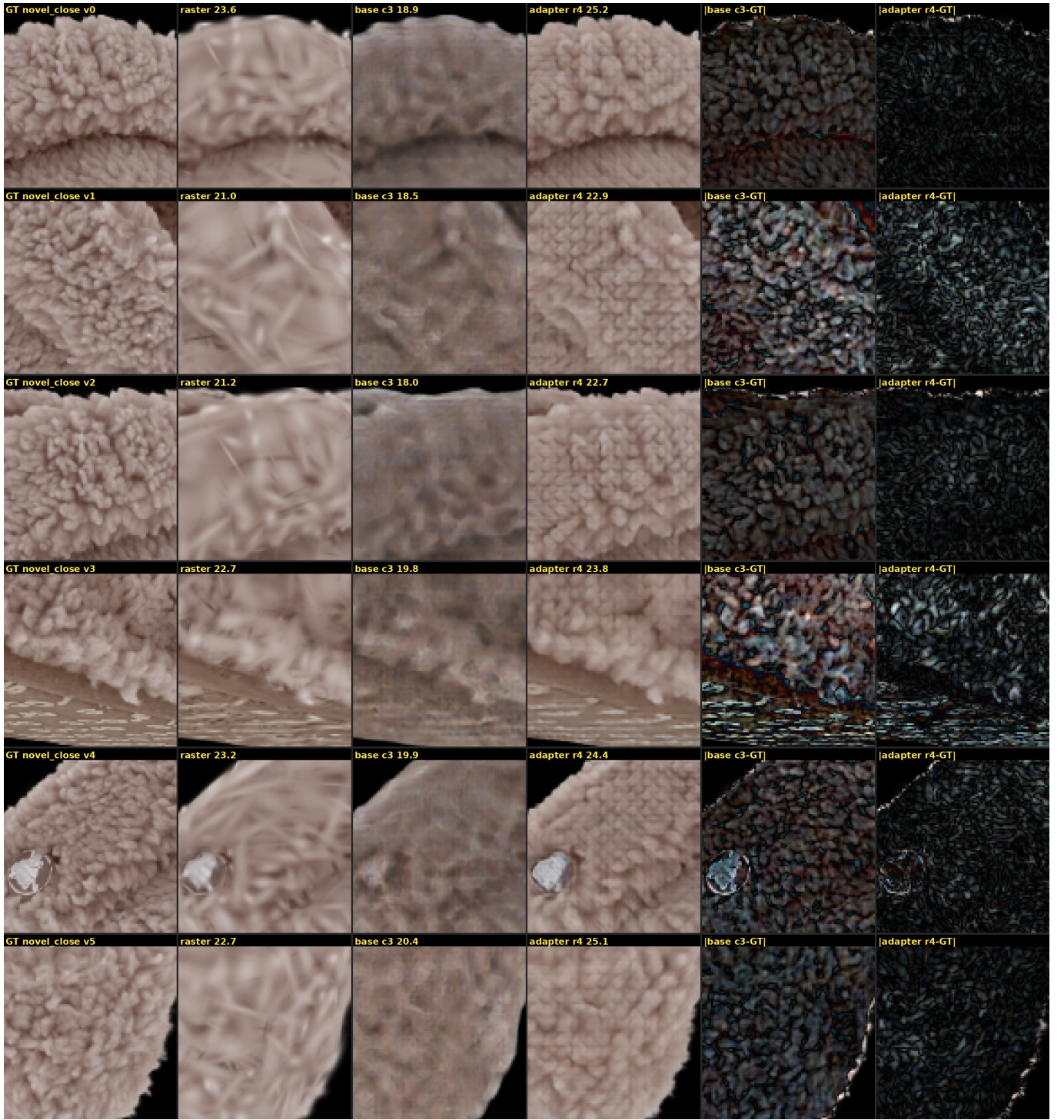


Figure 21: Slipper close range at native resolution, six views. The rasterizer smears the fleece into streaks; the base has the right colour and no texture; the adapter reconstructs the pile. The 8-px grid of section 4 is visible in the adapter column, clearest in view 1.

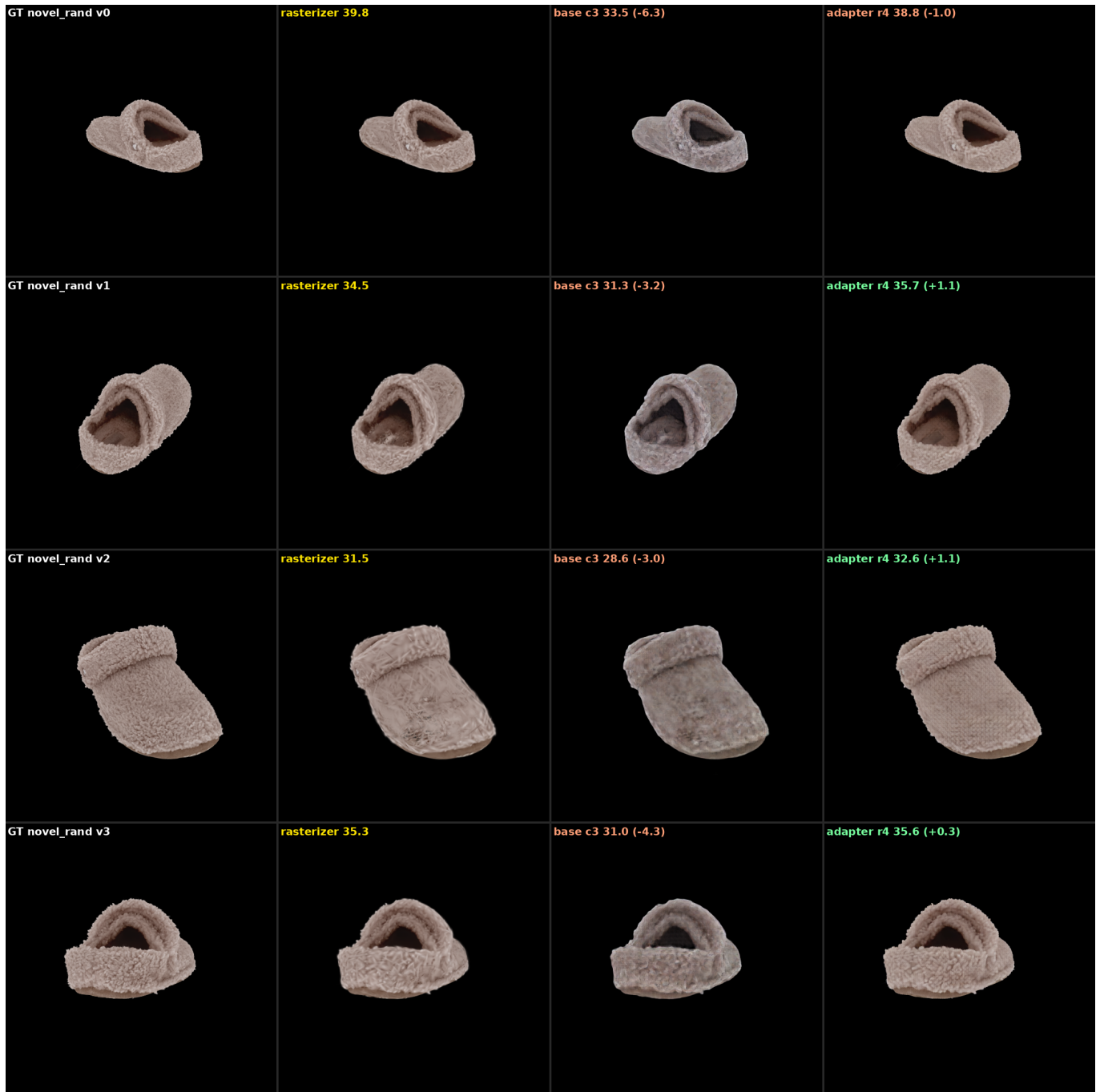


Figure 22: Slipper, random range, views 0-3: the adapter wins by about 1 dB on three of the four and loses on the one where the rasterizer already scores 40 dB.



Figure 23: Slipper, far range, views 0-1.

Which base matters. Adapter quality tracks the base’s held-out margin, not its fit, and the canvas base wins on both objects tried. On the molecule the rasterizer’s own bar is 9 dB higher (a simple object rasterizes near-perfectly), and there the adapter wins only the close range:

object	rasterizer PSNR	P2+fg base	P2+fg c2 base	canvas c3 base
slipper	35.8	-0.33	-0.44	+0.60 (30/40)
molecule	44.6	-2.70	-2.45	-1.41 (12/40), close +1.22 (7/8)

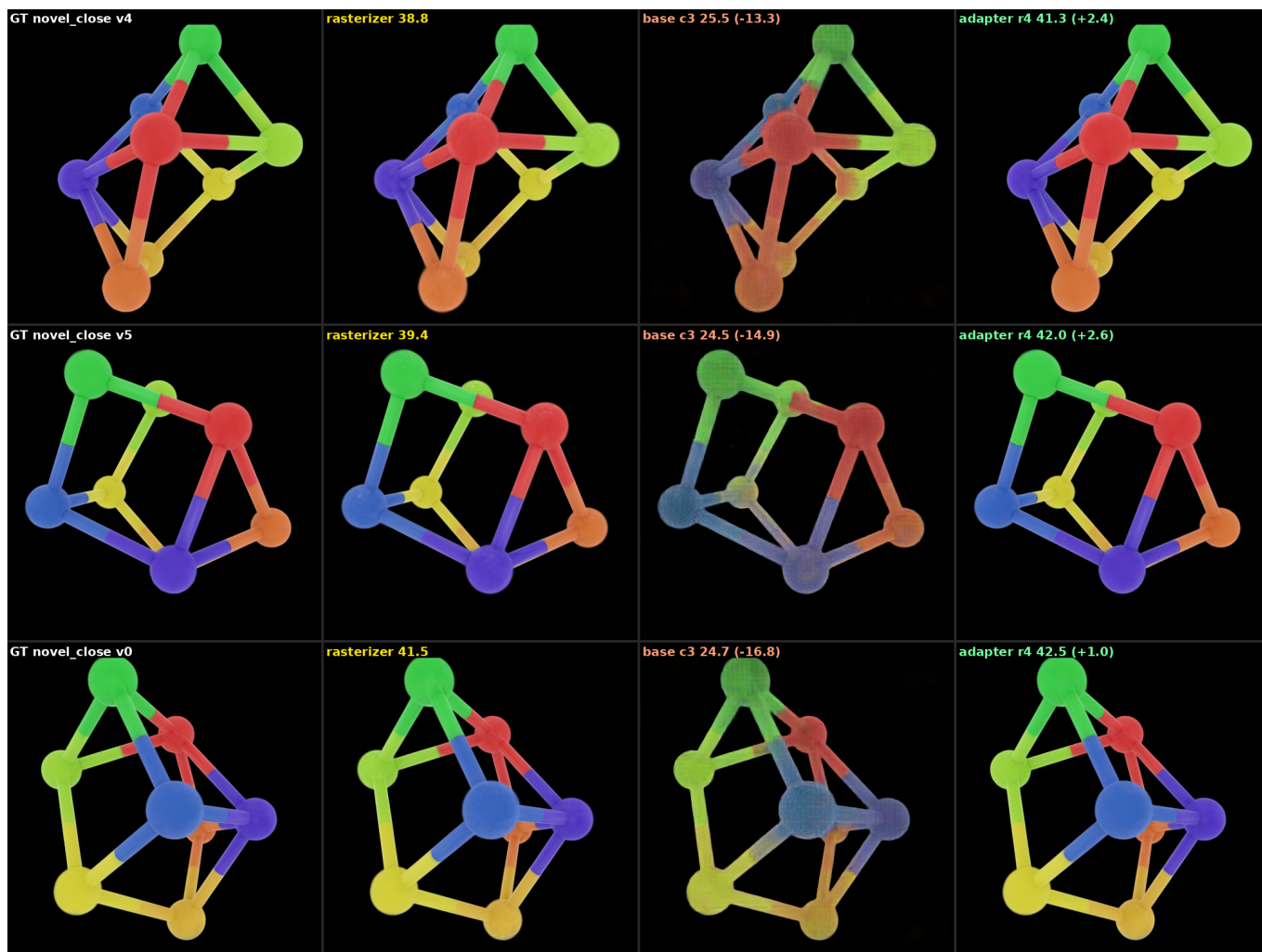


Figure 24: Molecule, close range: the two best views (+2.4, +2.6 dB) and view 0 (+1.0).

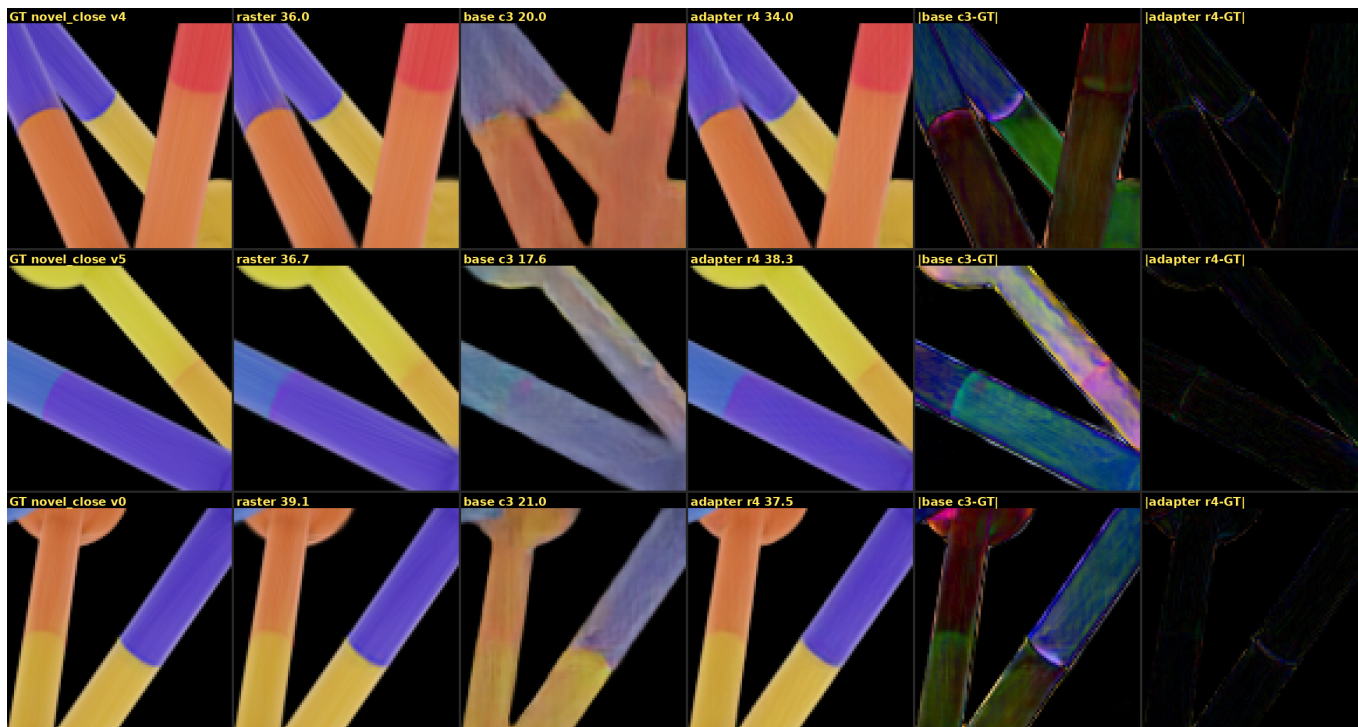


Figure 25: Molecule close range at native resolution: the adapter's ball surfaces are smoother than the rasterizer's but the bond edges are softer, and the grid is again present.

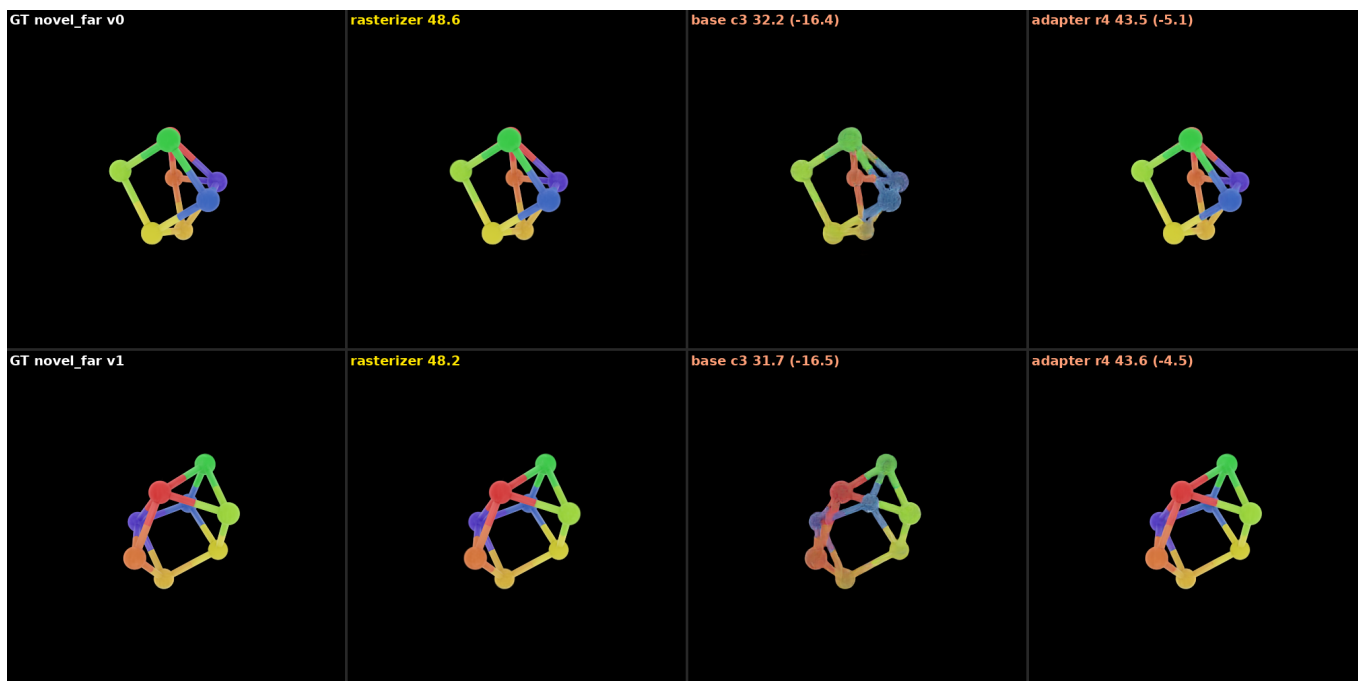


Figure 26: Molecule, far range: the rasterizer is at 48 dB and the adapter loses 4-5 dB to it. This is the object-dependence in one figure.

The win is object-dependent: it exists where the rasterizer is weak (fine texture, close range) and not where it is already near-perfect. The per-object full fine-tune reaches +1.82 dB on the slipper and +1.47 on the molecule, at 784.7 MB

and four to five cycles.

4. Three honest caveats

The rasterizer cannot be pruned after training. A black canvas at inference collapses the winning model below its own starting checkpoint; the right object from an adjacent camera is barely better. The canvas is consumed pixel-aligned, not as a generic prior.

canvas fed	fit	held-out	model PSNR
normal	-1.12	17.17	27.80
right object, wrong camera	27.80	29.42	15.54
all black (arm removed)	30.15	32.49	12.48

It is not copying the canvas. The control that *does* is shown below: the residual-over-canvas model (output = canvas + a zero-initialised 1x1 head on the decoder), which starts as the rasterizer and can only edit it. On the axe it scores 44.6 dB against the rasterizer’s 46.9 with the etchings intact, where our main-line model scores 28. But it never beats the rasterizer on any unseen object in any regime (2 to 5 dB below), and per-object adapter training makes it worse. Its number is the rasterizer’s minus a constant edit learned on the ten training objects, not a render.

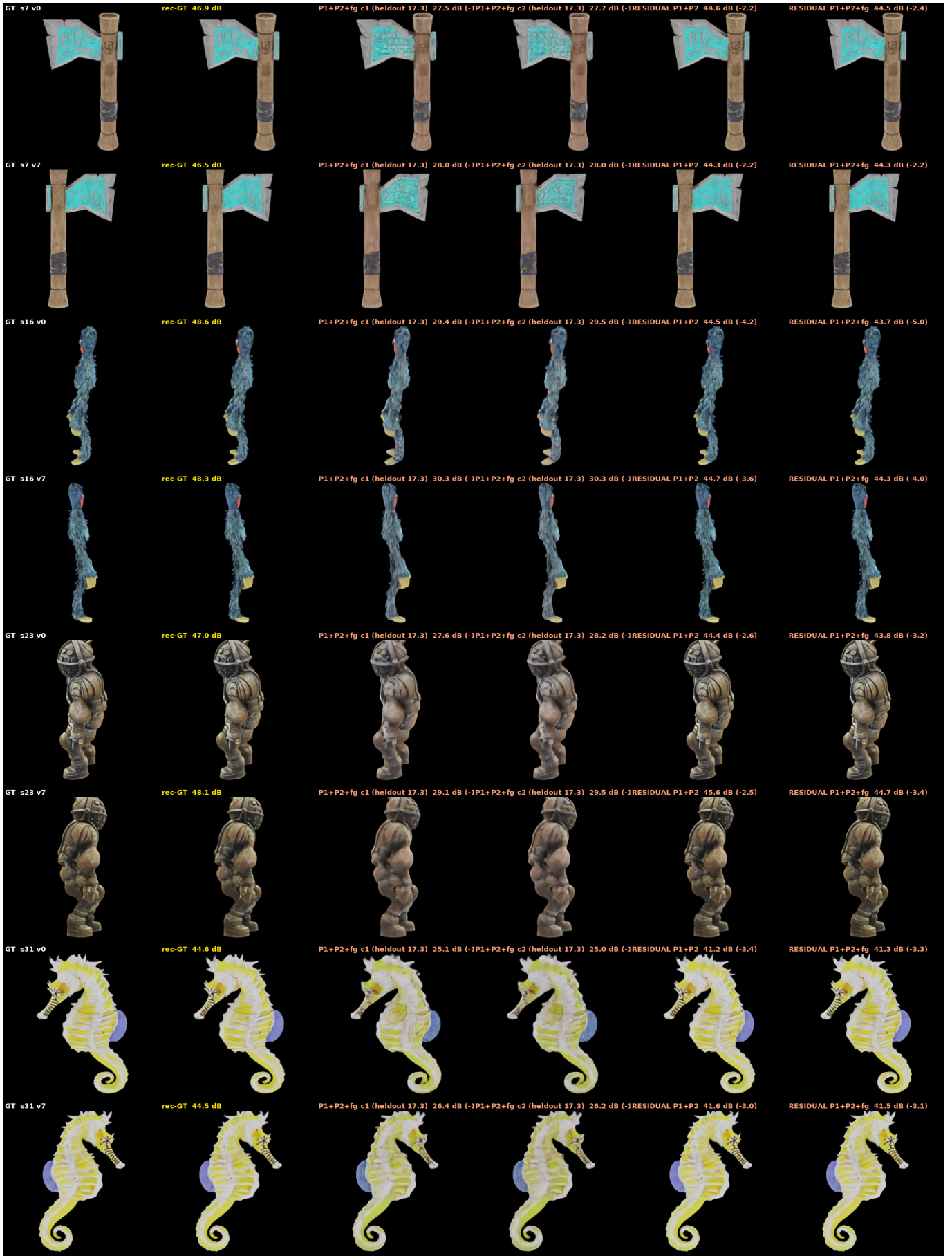


Figure 27: The residual-over-canvas control (two rightmost columns) next to the main-line canvas arm, on the same four unseen objects. It reproduces the rasterizer’s fine detail because it starts from it, and it never exceeds it.

On the main line, on unseen objects the output sits 18% away from the canvas it was given (27.5 dB from it); every pixel is synthesised. A control that adds its output to the canvas as a residual never beat the rasterizer on any unseen object and got worse with per-object training. Accurate description: **a learned, view-aligned refiner of a rasterization that synthesises every pixel it emits**, not a standalone neural renderer.

A patch-grid artefact survives, and only where we win. Period exactly 8 px, the patch size. Scored as error power at that frequency over its spectral neighbours (1.0 = none): base on its training views 1.00, base on 40 unseen objects 1.08, slipper adapter at random and far range 1.1-1.3, slipper adapter **close range 1.6-1.9**. It is visible in the slipper zoom rows above. A zero-initialised 9x9 residual convolution trained into the base did nothing (learned gain 1.2%), because the base’s own views carry no grid to learn from; the layer has to be trained in the adapter stage on the object’s close-range views.

5. Proposed next steps

1. **Rank-1 attention-only adapter on the canvas base** (0.48 MB). If it keeps most of the +0.60 dB this becomes a rate-distortion result. One day.
2. **Full-dataset run with the canvas recipe.** Justified by the N=10 to N=100 trend. About three days on one node.
3. **Train the deblock layer with the adapter**, on the object’s close-range views, where the grid lives. 3 KB, about 17 hours.
4. **Canvas dropout during training**, to keep the scene-token path alive and test whether the two paths can coexist.

Code: branches `exp/p1-canvas` (main line), `exp/p1-residual` (rejected control), `exp/p4-deblock` (this report). All numbers reproduce from `data_v10/probe_report.py`, `data_v10/probe_ci.py`; all renders from `data_v10/report_renders.py`.

Appendix: 40 unseen objects, rasterizer vs canvas c4

View 0 of the first 40 held-out objects, foreground-cropped. Left of each pair the rasterizer, right the canvas-c4 base, with crop PSNR.

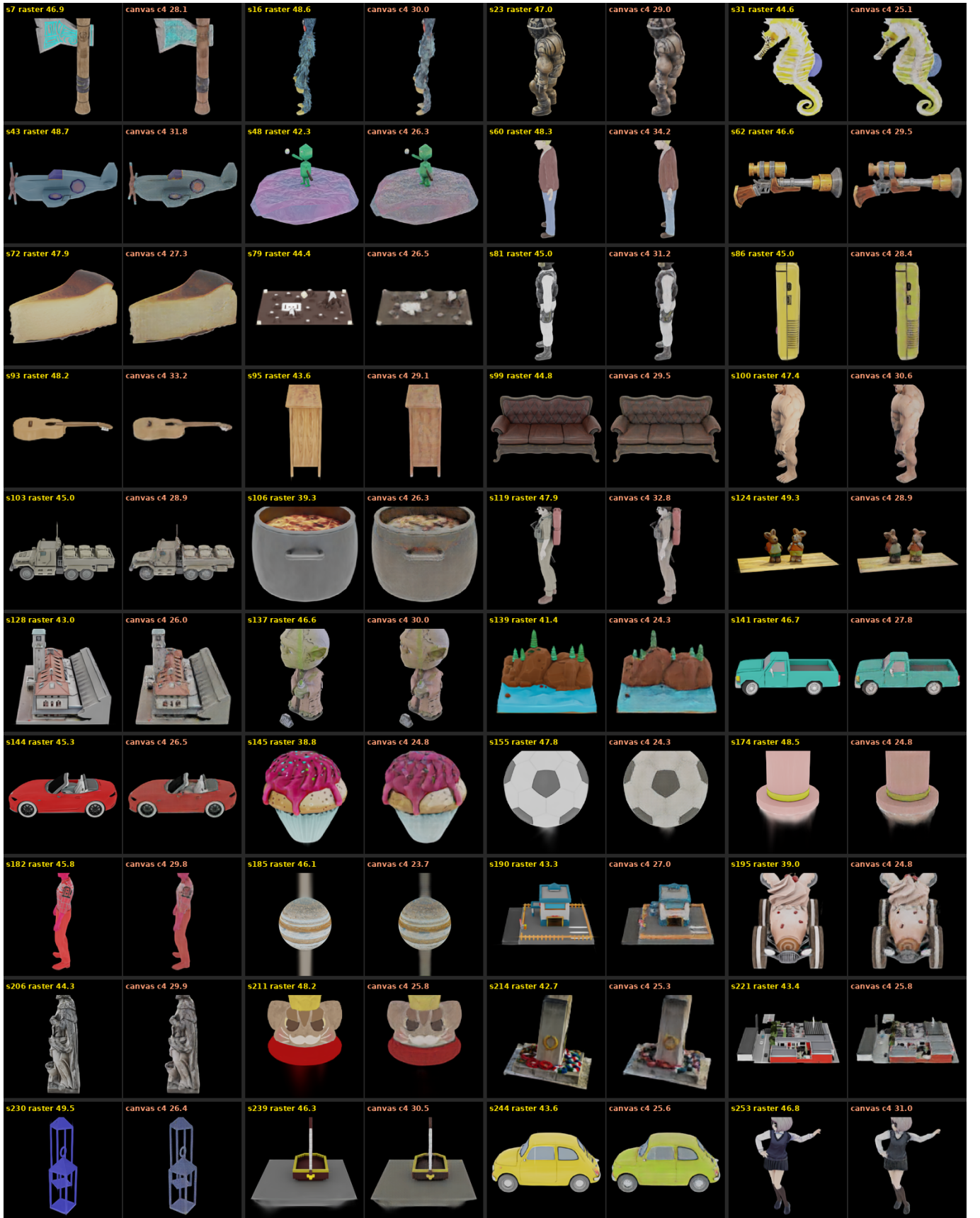


Figure 28: Rasterizer vs canvas-c4 base on 40 unseen objects, view 0, crop PSNR in the label.